# Where Apertus had difficulty

Team Claw & Order — Daniel Alznauer, Giuseppe Prezzo · Hack Apertus 2026, ZHAW challenge “See It, Say It, Pick It”

**Setup.** Apertus v1.5 8B (bfloat16, temperature 0, JSON-schema constrained output) reads one instruction and a text list of the objects in the photo: noun, colour, position, width in mm, what lies on what, which tray, and a hand. Widths come from one photo, which perspective enlarges, so the 85 mm gripper limit is applied with a factor of 1.3: a mug measured at 95 mm still counts as pickable. The list comes from open vision models (Grounding DINO, SAM 2.1, SigLIP 2) and plain geometry code. Apertus decides plan, ask or refuse, which object is meant, and the ordered actions. Code checks each reply against the measured scene and sends a specific error back, at most twice. Dev score 64.1 / 65. Beyond the 50 dev instructions we measured Apertus on 405 instructions of our own (same photos, new wording, one group per capability, written after development), on colour-shifted and blurred photos, and with misnamed objects.

## 1. It cannot ground objects reliably in the photo

Given the photo and asked for the pixel to pick, Apertus hit the right object in 9 of 40 instructions and refused 13 doable ones (9.9 / 65). Three of its pixels lay outside the image it saw, so it probably answers on a 0–1000 scale that its model card does not document; we read its pixels that way. Still far from usable: it sees roughly what is on the table, but calls gears “sun-shaped objects”, places things vaguely, says “nothing on top” where one object lies on another, and missed objects. Numbered marks on the image helped (15.8 grounding points) but stayed below a plain text list without the image (20.1). Here Apertus is a reasoner over facts, not a vision model: every number it uses comes from the vision models and the geometry code.

## 2. Spatial words: it does not use the measured facts it is given

- **Corrections are ignored.** When the check rejects a “closest to” or “left of” choice and lists the measured distances and the object that fits, the same error comes back in 98 % of the retries. Relations must be computed by code and checked; Apertus does not compute them from coordinates.
- **Destination read as selector.** “Put the mandarin on the plate” is read as “the mandarin that is on the plate”, and the instruction is refused. A dedicated correction message fixed all 12 test rows written for this.
- “**Left of” and “under”** cause most failed corrections: 8 of 24 “side” and 15 of 21 “under” instructions end in the code fallback. **Near-ties** (two gears 98 and 99 mm from a reference) make it ask instead of pick.

## 3. Refusals: false plans when the object does not exist

All unsafe instructions (move onto a hand, throw, pull out from under), all instructions with a hand in the photo and all pointing instructions (“that one”) were answered right. The weak spot is an object that does not exist: “pick up the yellow cube” where there is a red cube and a yellow gear. Apertus picks the yellow gear, choosing by colour over noun; on 56 new rows of this kind only 31 were refused. Two mechanisms made it worse: Apertus changed its own object kind from “cube” to “gear” after a correction, and when it kept refusing to the end, our fallback turned the refusal into a pick. Keeping Apertus’s final refusal raised the 56 rows to 40, with no doable instruction lost. A worked refusal example in the prompt (“pick up the red pen” where both pens are blue → refuse) added only 2 more (42 / 56, $p = 0.63$). A stricter colour check reached 52 / 56 but refused 39 more doable instructions on photos with warmer or cooler white balance, so we rejected it: refusing safely and not refusing too much pull against each other once colours are measured from pixels.

## 4. First replies rarely hold — and facts up front do not fix that

Only 39 % of first replies pass the checks (157 of 405). Corrections make up 39 % of the generated tokens on dev and 54 % on held-out. The most common reasons for a rejected first reply: selector words that are

not in the instruction (61), **picking an object while another still lies on it (58)**, the form of a refusal (37), an empty object kind (about 45), and a measured “closest” or “left of” that does not fit (27). The JSON schema prevents invented ids and broken JSON, not these.

The scene text always said “Lies under X”. We tested whether Apertus plans right when the consequence is spelled out as well: “*Lies under X: to pick it, first pick X and place it on the table.*” On the same 405 instructions, first replies that grab a covered object went from 58 to 54, within the noise (two runs of the same code differ by 8; exact McNemar p = 0.29); the total stayed at 387. Asked “could you hand me the red gear under the yellow gear”, with that sentence on the red gear’s own line, Apertus’s first plan is still only “pick red_gear_2”. **The first action list follows the verb of the instruction, not the facts of the scene**; Apertus applies the facts once the check names the violation. Showing instead of telling did not help either: the prompt already demonstrated the right plan for “put the bolt in the grey tray”, and 46 of the 58 errors came with the verb “pick”, so we added a worked example for “pick up the bolt”. The count stayed at 46. In short, extra prompt text — a fact, a demonstrated plan, a demonstrated refusal (section 3) — did not measurably change Apertus’s decisions; code that names the specific violation did. For an 8B model the check loop is not a convenience: it is what makes the plans physically possible.

## 5. It is sensitive to wording, serving mode and numerics

| Change | Effect |
|---|---|
| Rewording the system prompt without changing its meaning | 22 of 405 answers flip between right and wrong; only 152 of 405 replies stay byte-identical. We used this as the noise floor: a change had to beat it. |
| Docker instead of a local install, same code | Some replies change; no final decision changed. |
| 8 requests batched instead of one at a time | Answering 5.6× faster, but 3 of 50 dev decisions change (the same 3 in each of three runs); score 64.1 → 61.8. We kept one at a time. |
| float16 instead of bfloat16 | The activations overflow and every request fails. GPUs without bfloat16 (Turing) must use float32: twice the memory, slower. |

## What worked, and what we would do next

With measured facts in text form and a check that names violations, Apertus resolves referring expressions well: 387 of 405 of our own instructions right (weakest: absent colours 75 %, objects lying on top 78 %), 64.1 / 65 on dev, all unsafe and pointing instructions handled. The division of labour that worked: **vision models measure, code computes relations and checks, Apertus decides.** Of eleven changes judged by a rule fixed before the run, one was accepted.

**Next, accuracy: train, don’t prompt.** Prompt text did not change Apertus’s decisions; weights did. In a first test beyond this zero-shot challenge, a LoRA trained in 78 minutes on one GPU on 713 replies the check had accepted (7 photos) raised replies right the first time from 70 to 84 of 126 on three unseen photos (exact McNemar p = 0.0005, fewer corrections, 7 % less time), with final answers unchanged (122), but it traded 3 safety errors for 3 refusals of doable instructions. So we would train on the check’s rejected replies too (pairs of rejected and accepted replies), weight the safety rows, and use more photos; full fine-tuning only once LoRA stops improving and data from many cells exists. **Next, production: one small sovereign model per cell.** A quantised Apertus with the cell’s adapter on an 8 GB GPU next to the robot. Open is whether quantisation changes decisions, as batching did (64.1 → 61.8), so it would have to pass the same pre-set rule. Prerequisites for a real cell: a safety gate the model cannot override, a depth camera, and a re-check after every pick.

**A first try at cheaper corrections.** Asking a correction only for the reply fields from the first wrong one onwards cut correction text by 33 %, but held-out answering was only 5–8 % faster and more instructions ended in the fallback (core 18 → 29–31): Apertus then wrote “none” into the free-text selector field again and again. It fills a form better in full context than in fragments, so we kept the full rewrite.